Construiește un GPT de la zero

Ghid complet pentru proiectul tău: de la bigram la Transformer, și cum îl duci mai departe

Bazat pe tutorialul „Let's build GPT: from scratch, in code, spelled out” (Andrej Karpathy) și pe codul pe care l-ai scris în acest proiect. Explicațiile pornesc de la zero și folosesc exemple mici, calculate de mână.

Cuprins

1. Imaginea de ansamblu 3
2. Python și PyTorch: ce trebuie să stăpânești 4
3. Datele: din text în numere 5
4. Modelul bigram 6
5. Antrenarea 7
6. Self-attention, pas cu pas 8
7. De la un cap de atenție la GPT 9
8. Codul complet al unui GPT mic 10
9. Cum exploatezi proiectul mai departe 14
10. Erori frecvente și rezolvarea lor 16
11. Glosar 17
12. Resurse 17

1. Imaginea de ansamblu

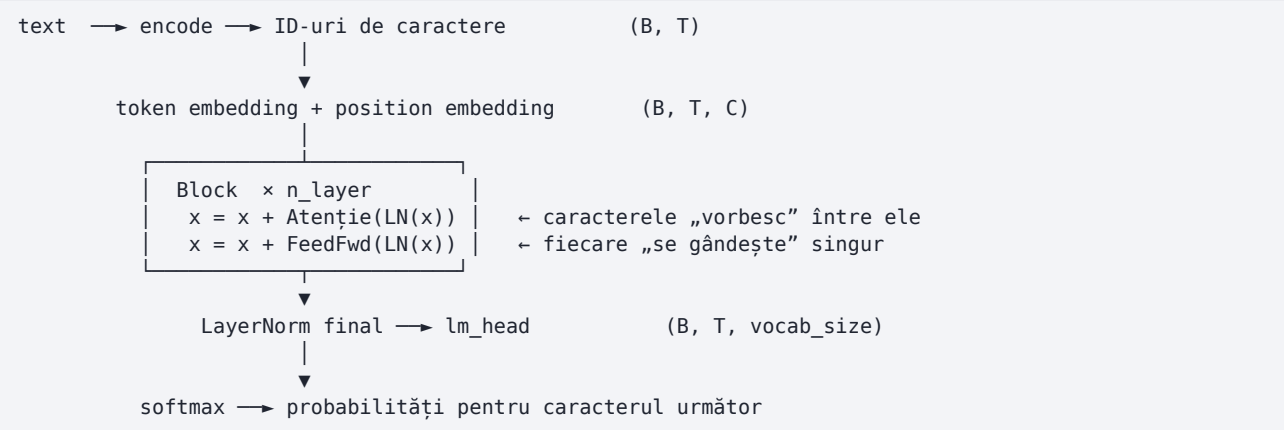
Un **model de limbaj** face un singur lucru: primește un text și ghicește ce urmează. Dacă îl pui să ghicească iar și iar, adăugând de fiecare dată ce a ghicit, obții text nou. ChatGPT, Claude și GPT-2 funcționează pe același principiu; diferă mărimea modelului, cantitatea de text și unitatea de lucru (bucăți de cuvinte în loc de caractere).

În proiectul tău, modelul lucrează **la nivel de caracter**: citește litere și ghicește litera următoare. E cea mai simplă variantă și cea mai bună pentru a înțelege mecanismul.

Drumul pe care l-ai parcurs

Etapă	Ce face modelul	Val loss tipic
Ghicit la întâmplare	Dă aceeași șansă celor 65 de caractere	≈ 4.17 (= ln 65)
Bigram	Prezice din litera curentă, cu un tabel	≈ 2.50
+ embeddings + poziții	Fiecare literă devine un vector de numere; știe și poziția	≈ 2.50
+ un cap de atenție	Literele se uită la literele dinaintea lor	≈ 2.40
+ multi-head, feed-forward, blocuri	Mai multe tipuri de atenție, prelucrare, adâncime	≈ 2.0 – 2.1
GPT complet (6 blocuri, 384)	Transformer decoder cu ~10.8 milioane de parametri	≈ 1.48

Schema unui GPT



Notația **(B, T, C)** apare peste tot: **B** = câte secvențe procesezi deodată (batch), **T** = câte poziții are fiecare secvență (time), **C** = câte numere descriu fiecare poziție (channels).

2. Python și PyTorch: ce trebuie să stăpânești

Python

Concept	Exemplu	Ce înseamnă
set / list / sorted	`sorted(list(set(text)))`	caracterele unice, într-o ordine fixă
enumerate	`for i, ch in enumerate(chars)`	dă poziția și elementul în același timp
dict comprehension	`{ch: i for i, ch in ...}`	construiește un dicționar într-o linie
list comprehension	`[stoi[c] for c in s]`	construiește o listă într-o linie
lambda	`encode = lambda s: ...`	funcție scurtă, cu o singură expresie
join	`''.join(lista)`	lipește o listă de caractere într-un text
slicing	`data[:n] data[n:] x[-8:]`	de la început până la n / de la n la final / ultimele 8
f-string	`f"loss {loss:.4f}"`	text cu valori inserate, aici cu 4 zecimale
clase	`class Head(nn.Module):`	o „rețetă” pentru un obiect cu date și funcții

Indentarea contează. Tot ce e împins spre dreapta sub `def` sau `class` le aparține. Codul care *folosește* o clasă sau o funcție se scrie înapoi la marginea din stânga. Cea mai frecventă eroare din proiect a fost cod pus din greșeală în interiorul unei funcții, după `return`.

Tensori

Un **tensor** e un container de numere cu oricâte dimensiuni, optimizat pentru calcule pe GPU și pentru calculul automat al derivatelor. Spre deosebire de o listă, operațiile se aplică pe toate numerele deodată.

```
t = torch.tensor([1, 2, 3])
t * 2                # tensor([2, 4, 6])   — o listă ar fi devenit [1,2,3,1,2,3]
t.shape              # torch.Size([3])
t.dtype              # torch.int64

torch.zeros(2, 3)    # matrice 2×3 plină de 0
torch.ones(3, 3)     # matrice 3×3 plină de 1
torch.randn(4, 8, 32) # numere aleatorii, forma (B=4, T=8, C=32)
torch.arange(5)      # tensor([0, 1, 2, 3, 4])
```

Operație	Ce face
`x.view(B*T, C)`	schimbă forma fără să schimbe numerele (turtește B și T într-o dimensiune)
`torch.stack([a, b])`	pune mai mulți tensori unul sub altul, creând o dimensiune nouă
`torch.cat([a, b], dim=-1)`	lipește tensori de-a lungul unei dimensiuni existente
`a @ b`	înmulțire de matrice
`k.transpose(-2, -1)`	inversează ultimele două dimensiuni: (B, T, hs) → (B, hs, T)
`torch.tril(torch.ones(T, T))`	triunghiul de jos: 1 pe diagonală și sub ea, 0 deasupra
`x.masked_fill(mask, v)`	pune valoarea v acolo unde masca e True
`x.to(device)`	mută tensorul pe GPU (sau CPU)
`x.item() / x.tolist()`	tensor cu un număr → număr Python / tensor → listă Python

dim=-1 înseamnă „ultima dimensiune”. La `F.softmax(wei, dim=-1)`, softmax-ul se aplică separat pe fiecare rând. Când ai o eroare, primul lucru de verificat e `.shape` la fiecare pas.

3. Datele: din text în numere

Tokenizare la nivel de caracter

```
chars = sorted(list(set(text)))       # toate caracterele distincte (65 la Shakespeare)
vocab_size = len(chars)
stoi = {ch: i for i, ch in enumerate(chars)}   # caracter -> număr
itos = {i: ch for i, ch in enumerate(chars)}   # număr -> caracter
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])

encode("hii")          # [46, 47, 47]
decode([46, 47, 47])   # 'hii'
```

Numerele nu au semnificație proprie; contează doar ca fiecare caracter să aibă mereu același număr unic. Sortarea garantează asta. **Tokenul** e unitatea cu care lucrează modelul; aici un token = un caracter.

Antrenare și validare

```
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]     # 90%: modelul învață din ele
val_data   = data[n:]     # 10%: nu le vede niciodată la antrenare
```

Validarea e „testul pe exerciții noi”. Dacă modelul merge bine pe train dar prost pe val, a memorat textul în loc să învețe tiparele limbii (**overfitting**).

Secvențe, ținte și batch-uri

O bucată de **block_size + 1** caractere conține **block_size** exemple de antrenare: după primul caracter urmează al doilea, după primele două urmează al treilea, și așa mai departe. De aceea `y` este `x` mutat cu o poziție:

```
x = [18, 47, 56, 57, 58,  1, 15, 47]        # "First Ci"
y = [47, 56, 57, 58,  1, 15, 47, 58]        # "irst Cit"
# context [18]               -> țintă 47
# context [18, 47]           -> țintă 56
# ...
# context [18, 47, ..., 47]  -> țintă 58
```

```
def get_batch(split):
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))  # poziții de start aleatorii
    x = torch.stack([data[i:i+block_size] for i in ix])        # (B, T)
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])    # (B, T), mutat cu 1
    return x.to(device), y.to(device)
```

Un batch de forma (4, 8) înseamnă 4 × 8 = 32 de exemple de antrenare procesate în paralel. Bucățile din batch sunt independente; nu „vorbesc” între ele.

4. Modelul bigram

Bigramul prezice caracterul următor **doar din caracterul curent**. Modelul este un singur tabel: un rând pentru fiecare caracter, iar în rând câte un scor pentru fiecare caracter care ar putea urma.

curent ↓ / următor →	a	b	n
a	0.1	0.2	**2.0**
b	**2.5**	0.1	0.3
n	**1.8**	0.1	0.2

Tabelul de mai sus e învățat din cuvântul „banana”: după **a** vine mereu **n**, după **b** și **n** vine **a**. Ordinea din alfabet nu contează; contează doar ce apare în textul de antrenare.

Logits, softmax, probabilități

Numerele din tabel se numesc **logits**: scoruri libere, pot fi și negative. Softmax le transformă în procente:

```
softmax(s_i) = e^(s_i) / suma tuturor e^(s_j)

scoruri  [ 0.1,  0.2,  2.0 ]
e^s      [ 1.11, 1.22, 7.39 ]    suma = 9.72
procente [ 11%,  13%,  76% ]
```

- scor mai mare → procent mai mare; ordinea se păstrează
- scoruri egale → procente egale
- scorul $-\infty$ → exact 0% (e la puterea $-\infty = 0$); de aici vine masca din atenție

Loss-ul (cross-entropy)

Loss-ul măsoară cât de „surprins” e modelul de răspunsul corect: **loss = −ln(probabilitatea dată caracterului corect)**, făcută media pe toate pozițiile.

Probabilitate dată răspunsului corect	Loss
100%	0
50%	0.69
10%	2.30
1/65 ≈ 1.5% (ghicit la întâmplare)	4.17

O interpretare utilă: $\mathbf{e^{loss}}$ ≈ între câte caractere „ezită” modelul. Loss 2.40 → $e^{2.4} \approx 11$ caractere plauzibile, în loc de 65.

Generarea

```
def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:]          # doar ultimele block_size caractere
        logits, _ = self(idx_cond)
        logits = logits[:, -1, :]                # doar ultima poziție
        probs = F.softmax(logits, dim=-1)        # scoruri -> probabilități
        idx_next = torch.multinomial(probs, 1)   # tragem la sorți după probabilități
        idx = torch.cat((idx, idx_next), dim=1)  # lipim caracterul nou
    return idx
```

`multinomial` trage la sorți, ca un zar măsluit. Dacă ai alege mereu caracterul cel mai probabil, textul ar fi mereu identic și s-ar bloca în bucle.

5. Antrenarea

```
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for step in range(max_iters):
    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)          # 1. predicție + cât de greșită e
    optimizer.zero_grad(set_to_none=True) # 2. ștergem derivatele vechi
    loss.backward()                       # 3. calculăm cum trebuie schimbat fiecare parametru
    optimizer.step()                      # 4. îi schimbăm puțin
```

- **Parametrii** sunt numerele din model care se învață (tabelele de embedding, greutățile din `nn.Linear`).
- **loss.backward()** folosește autograd: PyTorch calculează automat derivata loss-ului față de fiecare parametru (gradientul).
- **optimizer.step()** mută fiecare parametru puțin în direcția care scade loss-ul. AdamW e o variantă îmbunătățită de gradient descent.
- **learning_rate** = cât de mare e pasul. Prea mare: loss-ul sare sau devine NaN. Prea mic: învață foarte încet.
- **zero_grad** e necesar pentru că PyTorch adună gradienții de la un pas la altul dacă nu îi ștergi.

Evaluarea corectă a loss-ului

```
@torch.no_grad()              # doar măsurăm: nu calculăm derivate
def estimate_loss():
    out = {}
    model.eval()              # modul de evaluare (dezactivează dropout)
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            _, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()   # media pe mai multe batch-uri: un număr stabil
    model.train()
    return out
```

Cum citești curbele de loss

Ce vezi	Ce înseamnă	Ce faci
train și val scad împreună	modelul învață tipare reale	continuă / mărește modelul
train scade, val stagnează sau crește	overfitting: memorează textul	mai mult text, dropout mai mare, model mai mic, oprește mai devreme
amândouă rămân mari	underfitting: model prea mic sau antrenat prea puțin	mai mulți pași, model mai mare, context mai lung
loss-ul sare sau devine nan	learning rate prea mare	micșorează learning_rate (de ex. 1e-3 → 3e-4)

6. Self-attention, pas cu pas

Problema bigramului: vede doar ultimul caracter. Vrem ca fiecare poziție să adune informație de la **pozițiile dinaintea ei**, dar niciodată de la cele de după, pentru că la generare viitorul nu există încă.

Pasul 1: media trecutului

Exemplu cu cuvântul „cas”, fiecare literă descrisă prin 2 numere [e vocală?, e consoană?]:

```
x = [[0, 1],    # c
     [1, 0],    # a
     [0, 1]]    # s

media până la fiecare poziție:
  c -> [0, 1]
  a -> ([0,1] + [1,0]) / 2           = [0.5, 0.5]
  s -> ([0,1] + [1,0] + [0,1]) / 3   = [0.33, 0.67]
```

Pasul 2: aceeași medie, cu o înmulțire de matrice

```
tril = torch.tril(torch.ones(T, T))        wei = tril / tril.sum(1, keepdim=True)
[[1, 0, 0],                                [[1,    0,    0   ],
 [1, 1, 0],                                 [0.5,  0.5,  0   ],
 [1, 1, 1]]                                 [0.33, 0.33, 0.33]]

out = wei @ x     # același rezultat ca bucla, dar calculat într-o singură operație pe GPU
```

Rândul **t** din `wei` spune cât de mult se uită poziția t la fiecare altă poziție. Zerourile de deasupra diagonalei blochează viitorul.

Pasul 3: varianta cu softmax (scheletul atenției)

```
wei = torch.zeros((T, T))                          # scoruri de afinitate (deocamdată egale)
wei = wei.masked_fill(tril == 0, float('-inf'))    # viitorul -> −∞
wei = F.softmax(wei, dim=-1)                       # −∞ -> 0%, restul împărțit egal
out = wei @ x
```

```
[[0, -inf, -inf],   softmax pe rânduri   [[1,    0,    0   ],
 [0,  0,   -inf],   ──────────────────►   [0.5,  0.5,  0   ],
 [0,  0,    0  ]]                         [0.33, 0.33, 0.33]]
```

Dă tot media, dar funcționează și când scorurile **nu sunt egale**. Exact asta face atenția adevărată.

Pasul 4: scoruri învățate — query, key, value

```
key   = nn.Linear(C, head_size, bias=False)
query = nn.Linear(C, head_size, bias=False)
value = nn.Linear(C, head_size, bias=False)

k = key(x)      # (B, T, hs)  „ce sunt eu”
q = query(x)    # (B, T, hs)  „ce caut”
v = value(x)    # (B, T, hs)  „ce informație dau mai departe”

wei = q @ k.transpose(-2, -1) * head_size**-0.5   # (B, T, T) scorurile
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)
out = wei @ v                                     # (B, T, hs)
```

- **query**: fiecare poziție întreabă „ce caut?” (de ex. „o vocală dinaintea mea”).
- **key**: fiecare poziție răspunde „ce sunt?” (de ex. „sunt o vocală”).

- **q · k**: scor mare când întrebarea unei poziții se potrivește cu eticheta alteia.
- **value**: ce informație trimite efectiv o poziție celor care se uită la ea.
- *** head_size**-0.5** (împărțirea la √head_size): ține scorurile mici; altfel softmax ar da aproape 100% unei singure poziții.

> Exemplu: pentru poziția lui „s” din „cas”, modelul învață scorurile [−1, 2, 0.5]. Softmax dă [4%, 79%, 18%]: se uită mai ales la vocala „a”. Vectorul rezultat devine ≈ [0.79, 0.21] și „știe” că e o consoană după o vocală, informație utilă pentru a ghici că urmează tot o vocală (casa).

Notițe importante despre atenție

- Atenția nu cunoaște singură **ordinea**; de aceea adaugi `position_embedding_table`.
- Fiecare exemplu din batch e procesat separat; secvențele nu comunică între ele.
- Cu mască (`tril`) = **decoder** (GPT): bun pentru generare. Fără mască = **encoder** (BERT): toate pozițiile se văd între ele, bun pentru analiză sau clasificare.
- **Self**-attention: q, k, v vin din același x. **Cross**-attention: q dintr-o sursă, k și v din alta (de ex. la traducere).
- Poți privi atenția ca pe un graf în care fiecare nod (poziție) primește mesaje de la nodurile permise, ponderate de procentele din wei.

7. De la un cap de atenție la GPT

Multi-head attention

În loc de un singur cap de 32, folosești mai multe capete mici în paralel (de ex. 4 × 8). Fiecare poate învăța alt tip de relație (vocale, spații, început de cuvânt); rezultatele se lipesc cu `torch.cat` și trec printr-o proiecție liniară.

Feed-forward

Atenția **adună** informația; feed-forward o **prelucrează**. E un mic strat de neuroni aplicat fiecărei poziții separat: `Linear(C, 4C) → ReLU → Linear(4C, C)`.

Blocul Transformer

```
class Block(nn.Module):
    def forward(self, x):
        x = x + self.sa(self.ln1(x))     # comunicare
        x = x + self.ffwd(self.ln2(x))   # calcul
        return x
```

Ingredient	De ce e necesar
Conexiune reziduală (`x = x + f(x)`)	Fiecare bloc adaugă doar „ce e nou” peste informația existentă. Gradientul ajunge ușor până la primele straturi, așa că rețelele adânci se pot antrena.
LayerNorm	Normalizează vectorul fiecărei poziții (medie 0, variață 1), ca numerele să nu explodeze sau să dispară de la un strat la altul. GPT folosește varianta „pre-norm”: LN înainte de atenție și de feed-forward.
Dropout	La antrenare, „oprește” aleator o parte din neuroni (de ex. 20%), ca modelul să nu depindă de câțiva și să nu memoreze. Se dezactivează automat în `model.eval()`.
Mai multe blocuri	Fiecare bloc construiește peste cel dinainte: primele învață litere și silabe, următoarele cuvinte și structură.

Hiperparametrii modelului final din video

Parametru	Valoare mică (CPU)	Valoare finală (GPU)	Ce controlează
batch_size	16 – 32	64	câte secvențe pe pas
block_size	8 – 32	256	cât context vede modelul
n_embd	32 – 64	384	câte numere descriu un caracter
n_head	4	6	câte capete de atenție pe bloc
n_layer	3 – 4	6	câte blocuri
dropout	0.0	0.2	regularizare împotriva memorării
learning_rate	1e-3	3e-4	mărimea pasului de învățare
max_iters	5000	5000	câți pași de antrenare

Cu valorile finale, modelul are ≈ **10.8 milioane de parametri** și ajunge la val loss ≈ 1.48. Pe un GPU puternic durează câteva zeci de minute; pe procesor, ore. Pe un laptop fără GPU, folosește valorile mici sau Google Colab cu GPU.

8. Codul complet al unui GPT mic

Acesta e modelul final din tutorial, cu trei adăugiri utile: salvarea modelului, generarea dintr-un text de start (prompt) și controlul creativității (temperature, top-k). Codul a fost rulat și testat. Pe procesor, micșorează setările de sus (vezi tabelul din capitolul 7).

```
import urllib.request
import torch
import torch.nn as nn
from torch.nn import functional as F

# ---------------- setări (valorile mari din video, pentru GPU) ----------------
batch_size = 64       # câte secvențe procesăm deodată
block_size = 256      # câte caractere de context vede modelul
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4
eval_iters = 200
n_embd = 384          # câte numere descriu fiecare caracter
n_head = 6            # câte capete de atenție are fiecare bloc
n_layer = 6           # câte blocuri puse unul peste altul
dropout = 0.2
device = 'cuda' if torch.cuda.is_available() else 'cpu'
torch.manual_seed(1337)

# ---------------- date ----------------
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt",
    "input.txt")
with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()

chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for i, ch in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])

data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]


def get_batch(split):
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i + block_size] for i in ix])
    y = torch.stack([data[i + 1:i + block_size + 1] for i in ix])
    return x.to(device), y.to(device)


@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out
```

```
# ---------------- piesele Transformerului ----------------
class Head(nn.Module):
    """un cap de self-attention"""

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)                                        # (B, T, hs)
        q = self.query(x)                                      # (B, T, hs)
        wei = q @ k.transpose(-2, -1) * k.shape[-1] ** -0.5    # (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        v = self.value(x)                                      # (B, T, hs)
        return wei @ v                                         # (B, T, hs)


class MultiHeadAttention(nn.Module):
    """mai multe capete în paralel; rezultatele lor se lipesc"""

    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        return self.dropout(self.proj(out))


class FeedForward(nn.Module):
    """fiecare caracter își prelucrează singur informația adunată"""

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)


class Block(nn.Module):
    """un bloc Transformer: comunicare (atenție), apoi calcul (feed-forward)"""

    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
```

```
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))     # conexiune reziduală
        x = x + self.ffwd(self.ln2(x))   # conexiune reziduală
        return x


class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)                               # (B, T, C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))  # (T, C)
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)                                                # (B, T, vocab)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            loss = F.cross_entropy(logits.view(B * T, C), targets.view(B * T))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -block_size:]
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :] / temperature
            if top_k is not None:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = float('-inf')
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx


model = GPTLanguageModel().to(device)
print(sum(p.numel() for p in model.parameters()) / 1e6, 'milioane de parametri')

# ---------------- antrenare ----------------
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for it in range(max_iters):
    if it % eval_interval == 0 or it == max_iters - 1:
        losses = estimate_loss()
        print(f"step {it}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    xb, yb = get_batch('train')
```

```
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

# ---------------- salvare ----------------
torch.save(model.state_dict(), 'gpt_shakespeare.pt')

# ---------------- generare ----------------
prompt = "ROMEO:"
context = torch.tensor([encode(prompt)], dtype=torch.long, device=device)
print(decode(model.generate(context, max_new_tokens=500, temperature=0.8, top_k=40)[0].tolist()))
```

Încărcarea unui model salvat, fără reantrenare

```
model = GPTLanguageModel().to(device)
model.load_state_dict(torch.load('gpt_shakespeare.pt', map_location=device))
model.eval()
context = torch.tensor([encode("JULIET:")], dtype=torch.long, device=device)
print(decode(model.generate(context, max_new_tokens=300)[0].tolist()))
```

> Modelul salvat funcționează doar cu **aceleași setări** (n_embd, n_head, n_layer, block_size) și cu **același vocabular** (aceleași caractere în `chars`). Dacă schimbi textul de antrenare, trebuie să reantrenezi.

9. Cum exploatezi proiectul mai departe

9.1 Antrenează-l pe text în limba română

- **Cantitate:** un eseu de 5.000 de caractere e prea puțin; modelul doar îl memorează. Țintește la **cel puțin 1 MB de text** (≈ 1 milion de caractere), ideal mai mult.
- **Surse libere de drepturi:** autori clasici (Eminescu, Creangă, Slavici, Caragiale) de pe Wikisource în limba română; poți lipi mai multe opere într-un singur `input.txt`.
- **Curățenie:** scoate meniurile, numerele de pagină, spațiile multiple. Tot ce e în text, modelul învață.
- **Diacritice:** unifică formele cu sedilă (ş, ţ) și cele corecte cu virgulă (ș, ț). Altfel modelul le tratează ca litere diferite.

```
import re
text = (text.replace('ş', 'ș').replace('Ş', 'Ș')
            .replace('ţ', 'ț').replace('Ţ', 'Ț'))
text = re.sub(r'[ \t]+', ' ', text)       # spații multiple -> unul
text = re.sub(r'\n{3,}', '\n\n', text)   # maximum un rând gol între paragrafe
```

9.2 Controlează creativitatea la generare

Setare	Efect
`temperature < 1` (de ex. 0.7)	mai sigur, mai repetitiv, mai puține greșeli
`temperature > 1` (de ex. 1.3)	mai creativ, dar mai multe cuvinte inventate
`top_k = 40`	alege doar dintre cele mai probabile 40 de caractere; elimină alegerile absurde

Temperatura împarte logits-urile înainte de softmax: un număr mic accentuează diferențele, unul mare le aplatizează.

9.3 Experimente care te învață cel mai mult

Experiment	Ce să urmărești
Antrenează cu block_size 8, apoi 32, apoi 128	cum scade loss-ul când modelul vede mai mult context
Scoate position_embedding_table	modelul nu mai știe ordinea; loss-ul crește
Scoate masca tril	train loss scade spectaculos (modelul „trișează” uitându-se în viitor), dar generarea devine fără sens
Scoate conexiunile reziduale cu 6 blocuri	antrenarea devine mult mai lentă sau se blochează
n_layer 1, 2, 4, 6	cât câștigi din adâncime și cât costă în timp
dropout 0 vs 0.2 pe text puțin	diferența dintre train și val loss (overfitting)
Vizualizează wei	la ce caractere se uită fiecare poziție, pe un cuvânt real

```
# vizualizarea atenției primului cap din primul bloc
import matplotlib.pyplot as plt
head = model.blocks[0].sa.heads[0]
x = torch.tensor([encode("Ce mai faci?")], device=device)
pos = torch.arange(x.shape[1], device=device)
emb = model.token_embedding_table(x) + model.position_embedding_table(pos)
h = model.blocks[0].ln1(emb)
q, k = head.query(h), head.key(h)
wei = (q @ k.transpose(-2, -1) * k.shape[-1]**-0.5)
T = x.shape[1]
wei = wei.masked_fill(head.tril[:T, :T] == 0, float('-inf')).softmax(-1)
plt.imshow(wei[0].detach().cpu()); plt.colorbar(); plt.show()
```

9.4 Tokenizare pe bucăți de cuvinte (BPE)

Modelele reale nu lucrează pe caractere, ci pe **subcuvinte** (Byte Pair Encoding). Secvențele devin de câteva ori mai scurte, deci același block_size acoperă mult mai mult text. Prețul: vocabularul crește de la ~65 la zeci de mii, deci ai nevoie de mult mai mult text și de un model mai mare.

```
# pip install tiktoken
import tiktoken
enc = tiktoken.get_encoding("gpt2")      # vocabularul lui GPT-2: 50.257 de tokeni
ids = enc.encode("Luceafărul de Mihai Eminescu")
print(ids, len(ids))
print(enc.decode(ids))
# în proiect: encode = enc.encode, decode = enc.decode, vocab_size = enc.n_vocab
```

Tokenizerul GPT-2 a fost construit pe text în engleză, așa că textul românesc se rupe în mulți tokeni mici. Pentru română, un pas avansat este să antrenezi propriul tokenizer (de ex. cu biblioteca `sentencepiece` sau `tokenizers`).

9.5 Pasul profesionist: un model pre-antrenat + fine-tuning

Un model antrenat de la zero pe un laptop rămâne mic. În practică, se pornește de la un model deja antrenat pe mult text și se continuă antrenarea pe textul tău (**fine-tuning**). Biblioteca **Hugging Face Transformers** oferă modele GPT-2 gata antrenate; caută pe Hugging Face și modele GPT-2 antrenate pe limba română (de exemplu proiectul RoGPT2). Arhitectura lor e exact cea din acest ghid, doar mai mare.

9.6 Idei de proiecte

- Generator de text „în stilul lui Eminescu” sau al lui Creangă.
- Comparație între model pe caractere și model pe BPE, pe același text: loss, viteză, calitate.
- Raport de laborator cu curbele de loss pentru diferite valori ale block_size, n_layer și dropout.

- Vizualizarea hărților de atenție pe cuvinte românești și interpretarea lor.
- Varianta encoder (fără mască) folosită pentru clasificare de text, de exemplu „poezie sau proză”.

10. Erori frecvente și rezolvarea lor

Eroare / simptom	Cauză	Soluție
Codul de după o funcție nu afișează nimic	e indentat în interiorul funcției, după return	adu-l la marginea din stânga
`NameError: name 'vocab_size'`	variabila nu a fost creată în sesiunea curentă	rulează celulele de sus; în Colab: Runtime → Run all
`SyntaxError` la `!wget`	`!` există doar în Colab/Jupyter	folosește `urllib.request.urlretrieve(url, "input.txt")`
`wget: invalid option -- 0`	cifra 0 în loc de litera O	`!wget -O input.txt URL`
`No module named 'torch'`	scriptul rulează cu alt Python decât cel care are torch	verifică interpretorul (prima linie din consolă), deschide proiectul corect; `pip install torch` în acel mediu
`KeyError` în encode	caracter care nu apare în textul de antrenare	folosește doar caractere din `chars` sau normalizează textul
`IndexError: index out of range in self`	la generare, secvența depășește block_size	`idx_cond = idx[:, -block_size:]`
`RuntimeError: shape` / `mat1 and mat2`	dimensiuni care nu se potrivesc	afișează `.shape` la fiecare pas și compară cu (B, T, C)
`Expected all tensors to be on the same device`	modelul pe GPU, datele pe CPU (sau invers)	`.to(device)` pe model și pe date
`CUDA out of memory`	model sau batch prea mare pentru placa video	micșorează batch_size, block_size sau n_embd
Textul afișat ca listă de litere	lipsește `''.join` în decode	`decode = lambda l: ''.join([itos[i] for i in l])`
wget salvează input.txt.1, .2 ...	fișierul există deja	folosește `-O input.txt` ca să-l suprascrie

11. Glosar

Termen	Explicație
Token	unitatea cu care lucrează modelul: aici un caracter; la GPT-2 o bucată de cuvânt
Vocabular (vocab_size)	toate tokenurile posibile
Embedding	vectorul de numere învățat care reprezintă un token sau o poziție
Logits	scorurile brute, înainte de softmax
Softmax	transformă scorurile în probabilități care adună 1
Loss / cross-entropy	−ln(probabilitatea răspunsului corect), făcută media; cu cât e mai mic, cu atât mai bine
Gradient	cum se schimbă loss-ul dacă modifici puțin un parametru
Backpropagation	calculul gradienților de la loss înapoi prin rețea (`loss.backward()`)
Optimizer (AdamW)	algoritmul care actualizează parametrii folosind gradienții
Learning rate	mărimea pasului de actualizare
Batch	grupul de secvențe procesate deodată
Context / block_size	câți tokeni din trecut vede modelul
Self-attention	mecanismul prin care fiecare poziție adună informație de la alte poziții, cu ponderi învățate
Query / Key / Value	„ce caut” / „ce sunt” / „ce dau mai departe”
Mască cauzală	matricea triunghiulară care blochează viitorul
Head / multi-head	un mecanism de atenție / mai multe în paralel
Feed-forward	rețea mică aplicată fiecărei poziții separat
Conexiune reziduală	x = x + f(x): stratul adaugă peste informația existentă
LayerNorm	normalizarea vectorului fiecărei poziții
Dropout	oprirea aleatoare a unor neuroni la antrenare, împotriva memorării
Overfitting	modelul memorează datele de antrenare și merge prost pe date noi
Temperature / top-k	controlul creativității la generare
Pre-training / fine-tuning	antrenare pe text mult și general / continuarea antrenării pe textul tău
Decoder / encoder	cu mască, pentru generare (GPT) / fără mască, pentru analiză (BERT)

12. Resurse

- **Video:** Andrej Karpathy, „Let's build GPT: from scratch, in code, spelled out” (YouTube).
- **Codul din video:** github.com/karpathy/ng-video-lecture și versiunea extinsă github.com/karpathy/nanoGPT.
- **Seria de bază:** Karpathy, „Neural Networks: Zero to Hero” (micrograd, makemore), pentru backpropagation și embeddings explicate de la zero.
- **Articolul original:** Vaswani et al., „Attention Is All You Need” (2017), arxiv.org/abs/1706.03762.
- **Vizual:** Jay Alammar, „The Illustrated Transformer” și „The Illustrated GPT-2”.
- **Documentație:** pytorch.org/docs (nn.Embedding, nn.Linear, F.cross_entropy, torch.optim.AdamW).

- **Tokenizare:** Karpathy, „Let's build the GPT Tokenizer” (YouTube); biblioteca tiktoken.

> **Ordinea recomandată:** termină video-ul cu multi-head, feed-forward și blocuri în codul tău; rulează modelul complet pe Colab cu GPU; apoi treci pe text românesc de cel puțin 1 MB și fă experimentele din capitolul 9.3.